

Day 3 – Java Fundamentals

Basics, Input Handling, Operators, Methods & Practice Programs

1. Introduction to Java

Java is a high-level, object-oriented programming language that is designed to be platform-independent and is widely used for developing software applications. It allows users to write programs that can run on different platforms using the **JVM (Java Virtual Machine)**.

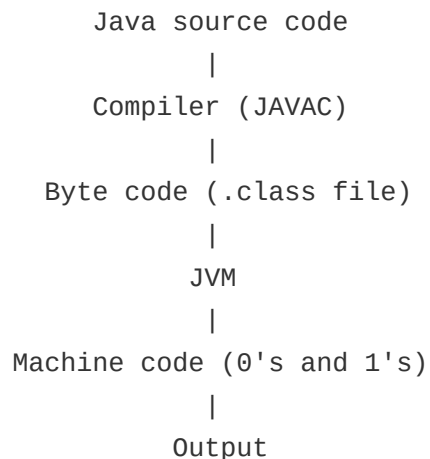
Program 1: Print "Hello Guys"

```
class Main {  
    public static void main(String[] args) {  
        System.out.println("Hello Guys...");  
    }  
}
```

2. Understanding public static void main()

- **public** – access modifier / specifier.
- **static** – normally, to access a class member we need to create an object. The keyword **static** means we don't need to create an object to call the method.
- The **JVM** can directly call `main()` using the class – without creating an object.

Java Compilation & Execution Flow



3. Taking Input Using Scanner

Program 2: Reading Name, Age, CGPA & Gender

```
import java.util.Scanner;
```

```
class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        System.out.print("Enter ur name: ");
        String name = sc.nextLine();

        System.out.print("Enter ur age: ");
        int age = sc.nextInt();

        float cgpa = sc.nextFloat();
        char gender = sc.next().charAt(0);

        System.out.println("Name: " + name);
        System.out.println("Age: " + age);
        System.out.println("CGPA: " + cgpa);
        System.out.println("Gender: " + gender);
    }
}
```

Output:

```
Enter ur name: mahalaxmi
Enter ur age: 20
9.8
F
Name: mahalaxmi
Age: 20
CGPA: 9.8
Gender: F
```

4. Operators in Java

Arithmetic Operators

- +
- -
- *
- %
- /
- ** (power)

Logical Operators

- AND (&&)
- OR (||)
- NOT (!)

Relational Operators

- == (equal to)
- != (not equal to)

Other Operator Categories

- Assignment Operator – =
- Unary Operators
- Ternary Operator

Program 3: Ternary Operator – Check Majority

```
import java.util.Scanner;

class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int age = sc.nextInt();

        String res = (age >= 18) ? "major" : "minor";
        System.out.print(res);
    }
}
```

Sample Runs:

```
age = 16 -> minor
age = 20 -> major
```

5. Methods in Java

A method in Java is a block of code that performs a specific task. It can be called wherever we need that task performed. Types of methods:

- In-built methods
- User-defined methods
- Recursive methods

Program 4: A Simple add() Method

```
class Main {
    public static int add(int a, int b) {    // function definition
        int c = a + b;
        return c;                          // return statement
    }

    public static void main(String[] args) {
        int a = 10;
        int b = 20;
        System.out.print(add(a, b));        // function call
    }
}
```

```
}
```

6. Quick Reference: STL (C++)

STL stands for the **Standard Template Library** in C++. It is organized around three main ideas:

- **Containers** – to store data. e.g. vector, list, stack, etc.
- **Algorithms** – to perform operations. e.g. sort(), reverse(), find(), max(), etc.
- **Iterators** – used for traversal.

7. Practice Problem: Plus One

Given an array of digits representing a non-negative integer, add one to the number represented by the digits.

Program 5: Plus One

Example trace: digits = [9, 9, 9] → each 9 rolls over to 0, so a new array is created → result = [1, 0, 0, 0].

```
class Solution {
    public int[] plusOne(int[] digits) {
        int n = digits.length;

        for (int i = n - 1; i >= 0; i--) {
            if (digits[i] < 9) {
                digits[i]++;
                return digits;
            }
            digits[i] = 0;
        }

        int[] res = new int[n + 1];
        res[0] = 1;
        return res;
    }
}
```

8. Pattern Printing Programs

Program 6: Star Triangle Pattern

```
class Main {
    public static void main(String[] args) {
        int n = 4;
        for (int i = 1; i <= n; i++) {
            for (int j = 0; j < i; j++) {
                System.out.print("* ");
            }
            System.out.println();
        }
    }
}
```

```
}
```

Output:

```
*
* *
* * *
* * * *
```

Program 7: Bordered Square Pattern

```
class Main {
    public static void main(String[] args) {
        int n = 4;
        for (int i = 1; i <= n; i++) {
            for (int j = 1; j <= n; j++) {
                if (i == 1 || j == 1 || i == n || j == n) {
                    System.out.print("* ");
                } else {
                    System.out.print("  ");
                }
            }
            System.out.println();
        }
    }
}
```

Output:

```
* * * *
*       *
*       *
*       *
* * * *
```

Program 8: Number Triangle Pattern

```
class Main {
    public static void main(String[] args) {
        int n = 4;
        for (int i = 1; i <= n; i++) {
            for (int j = 1; j <= i; j++) {
                System.out.print(j);
            }
            System.out.println();
        }
    }
}
```

Output:

```
1
1 2
1 2 3
1 2 3 4
```

9. Factorial Programs

Program 9: Factorial Using a For Loop

```
class Main {
    public static void main(String[] args) {
        int n = 4;
        int fact = 1;

        for (int i = 1; i <= n; i++) {
            fact = fact * i;
        }

        System.out.print(fact);
    }
}
```

Output:

24

Program 10: Strong Number Checker (Using a Factorial Method)

A **strong number** is a number whose sum of the factorials of its digits equals the number itself (e.g. $145 = 1! + 4! + 5!$).

```
class Main {
    public static int fact(int n) {
        int fact = 1;
        for (int i = 1; i <= n; i++) {
            fact = fact * i;
        }
        return fact;
    }

    public static void main(String[] args) {
        int n = 145;
        int sum = 0;
        int org = n;

        while (n != 0) {
            int d = n % 10;
            sum = sum + fact(d);
            n = n / 10;
        }

        if (sum == org) {
            System.out.print("Strong number");
        } else {
            System.out.print("Not a strong number");
        }
    }
}
```

Output:

Strong number

10. Arrays - Introduction

An **array** is a collection of multiple values of the same data type stored under a single variable name.

A **2D array** is also known as a **matrix** – its rows, columns, and elements are arranged like a table.